

Todo 프로젝트 — Phase 1 "데이터 계층 + CRUD API" 완전 학습 정리

이 문서의 목적: Phase 0(배포 기반) 위에, EF Core + PostgreSQL로 데이터를 저장하고 CRUD API를 만들어 배포까지 한 Phase 1의 전 과정을, 신입도 이해할 수 있게 용어·단계·겪은 예러·흔한 예러까지 담아 정리한다.

핵심 원칙: 면접에서 끝까지 설명 못 하는 건 안 적는다. 모든 코드·결정은 직접 손으로 실행하고 이해한 것만 담았다.

0. 한 줄 요약 — Phase 1에서 한 일

C# 클래스로 테이블을 정의(엔티티) → EF Core가 그걸 실제 PostgreSQL 테이블로 생성(마이그레이션) → 그 테이블에 데이터를 넣고 빼는 CRUD API 작성 → 비밀(연결문자열)을 환경별로 분리해 배포.

Phase 0이 "빈 앱을 배포하는 길"을 뚫은 거라면, Phase 1은 그 앱에 진짜 데이터베이스와 기능을 채운 단계다.

1. 큰 그림 — Phase 1 데이터 흐름

```
[클라이언트] → [Controller] → [DbContext] → [PostgreSQL]
HTTP 요청  CRUD 처리  EF Core 번역  tododb
(JSON)    DTO 검증   LINQ → SQL   Users/Todos 테이블
```

핵심 구성요소: - **엔티티(Entity)**: 테이블에 대응하는 C# 클래스 (User, Todo) - **DbContext**: 엔티티 ↔ 테이블 매핑 + DB 통신 관문 - **마이그레이션**: 엔티티 변경을 DB에 반영하는 "공사 지시서" - **DTO**: API가 외부와 주고받는 전용 그릇 - **컨트롤러**: HTTP 요청을 받아 CRUD 처리

2. 핵심 용어 사전 (신입용)

2-1. EF Core / ORM

용어	쉬운 뜻
ORM	객체(C# 클래스)와 관계형 DB(테이블)를 자동으로 이어주는 기술. SQL을 직접 안 써도 됨
EF Core	.NET의 대표 ORM (Entity Framework Core)
Npgsql	EF Core가 PostgreSQL과 통신하게 해주는 드라이버(provider)
엔티티(Entity)	테이블 한 개에 대응하는 C# 클래스
DbContext	엔티티들을 모아 DB와 통신하는 중앙 클래스. "이 DB에 어떤 테이블이 있다"는 지도
DbSet	DbContext 안에서 테이블 하나를 나타내는 속성 (<code>DbSet<Todo> Todos</code>)
Code First	코드(엔티티)를 먼저 만들고, 그걸로 DB를 생성하는 방식. SSMS에서 테이블 먼저 만들던 것과 반대
마이그레이션(Migration)	엔티티의 현재 모습을 DB에 반영하는 SQL 설계도 (변경 이력)
change tracking	EF Core가 가져온 엔티티의 변경을 추적했다가 SaveChanges 때 반영하는 기능

2-2. API / 웹

용어	쉬운 뜻
DTO	Data Transfer Object. API가 외부와 데이터를 주고받는 전용 그릇 (엔티티를 직접 노출 안 함)
over-posting	클라이언트가 보내면 안 되는 필드(id 등)를 보내 덮어쓰는 공격. DTO로 막음
컨트롤러(Controller)	HTTP 요청을 받아 처리하는 클래스 묶음
DI (의존성 주입)	필요한 객체를 직접 만들지 않고 외부(컨테이너)에서 주입받는 패턴
record	데이터 운반용 C# 타입. 불변(immutable) + 값 기반 동등성. DTO에 적합
async/await	비동기 처리. DB 대기 중 스레드를 안 잡아 처리량 ↑
LINQ	C#에서 컬렉션·DB를 질의하는 문법 (<code>.Where().Select()</code> 등)

2-3. 인증/설정 (Phase 1에서 맞본 것)

용어	쉬운 뜻
User Secrets	로컬 개발용 비밀 저장소. 프로젝트 밖에 저장돼 git에 안 올라감
연결문자열	DB 접속 정보(host/port/db/user/password/ssl) 한 줄
환경변수	실행 환경에 주입하는 설정값. 클라우드에선 이걸로 연결문자열 전달
Container App secret	Azure Container App에 저장하는 비밀. 환경변수가 이걸 참조(secretref)

3. DB 호스팅 결정 (왜 Azure를 골랐나)

선택지	무료 조건	외부 접속	결정
Azure Flexible Server	무료 계정 12개월, 매월 750h B1ms + 32GB	방화벽 규칙 필요	선택
Neon	영구 무료(소규모), scale-to-zero	기본 제공	후보
Supabase	무료, 7일 미사용 시 일시정지	기본 제공	후보

Azure를 고른 이유: AZ-204/DP-300 자격증 트랙과 한 생태계로 묶이고, 방화벽·SSL·백업 설정이 그대로 "운영 깊이" 포트폴리오 + DP-300 실습이 됨. VM 불필요(완전관리형 PaaS).

핵심 구조 — 서버 1대 공유, DB만 분리:

```
pg-shared-dglee (서버 1대, rg-deploy-test 소속 = 공용 인프라)
├── tododb (Todo용)
└── reservationdb (나중에 03용)
```

- 무료 한도(750h/월)는 **서버 단위** → 한 대 안에 DB 여러 개는 추가 비용 없음
- 매월 750시간씩 12개월 → 사실상 한 대를 1년간 풀가동 무료
- 공용 자원이므로 ACR·환경처럼 **rg-deploy-test** 에 배치 (일관성)

4. 단계별 상세 (1 ~ 10)

1단계 — PostgreSQL Flexible Server 생성

Azure Portal 또는 CLI로 생성. 핵심 설정: - 티어: **Burstable / Standard_B1ms** (무료 한도 사양) - 스토리지: **32GiB**, 자동 증가 끄 - 고가용성: 사용 안 함, 지역 중복: 끄 - 버전: PostgreSQL 18 (16 의도였으나 18로 생성됨 — CRUD엔 차이 없어 유지)

무료 비용 표시 함정: 생성 화면 예상 비용이 ~\$23/월로 떠도, "최대 750시간 무료 / 최대 32GB까지 무료"가 초록색으로 표기됨 → 무료 계정이면 차감되어 실제 0원. 표시는 정가 기준.

2단계 — 방화벽 설정

- ☒ "Azure 내 모든 서비스의 이 서버 액세스 허용" — app-todo(Container App)가 붙는 통로
- ☒ "현재 클라이언트 IP 추가" — 내 PC에서 접속용
- ☒ **AllowAll** (0.0.0.0~255.255.255.255)은 삭제 — 인터넷 전체 개방은 위험

중요 개념: 채용 담당자는 DB에 직접 접속하지 않는다. 담당자는 앱 URL(웹/Swagger)만 본다. DB는 인터넷에 직접 노출하지 않고 앱 계층을 통해서만 접근하게 하는 것이 정석. → 담당자 IP를 열 필요 없음.

3단계 — 로컬 접속 확인 (VS Code PostgreSQL 확장)

- 확장: Microsoft 공식 **ms-ossdata.vscode-pgsql**
- 접속 정보: Host(엔드포인트) / Port 5432 / User pgadmin / Password / Database **postgres** / **SSL Mode require**
- SSL은 Azure PostgreSQL 필수. Advanced에서 require 확인.

DB들 개념: SSMS(MSSQL), Toad(Oracle)에 해당하는 게 PostgreSQL에선 DBeaver/pgAdmin. 단, 우리는 VS Code 확장으로 대체해 별도 설치 불필요.

4단계 — EF Core 패키지 설치

```
dotnet add TodoApi package Npgsql.EntityFrameworkCore.PostgreSQL --version 9.0.4
dotnet add TodoApi package Microsoft.EntityFrameworkCore.Design --version 9.0.0
dotnet tool install --global dotnet-ef --version 9.*
```

- 버전 **family**를 9로 통일이 핵심: .NET 9 → EF Core 9 → provider 9.x → dotnet-ef 9.x
- provider 10.x는 .NET 10용 (메이저 어긋나면 문제)

5단계 — 엔티티 + DbContext (Phase 1 심장부)

User 엔티티 (**Models/User.cs**): Id, Email, PasswordHash, DisplayName, CreatedAt, **List<Todo> Todos** (네비게이션)

Todo 엔티티 (**Models/Todo.cs**): Id, Title, Description, **Status(enum 3단계: NotStarted/InProgress/Completed)**, **Priority(enum: Low/Medium/High)**, DueDate, CreatedAt, UpdatedAt, UserId(FK), User(네비게이션)

DbContext (**Data/AppDbContext.cs**):- **DbSet<User>**, **DbSet<Todo>** — 각각 테이블 - **OnModelCreating** 에서 매핑 규칙:- **HasIndex(Email).IsUnique()** — 이메일 중복 방지 - **HasMaxLength** — 컬럼 길이 제한 - **HasConversion<string>()** — enum을 숫자 대신 문자열로 DB 저장 - **HasOne/WithMany/HasForeignKey/OnDelete(Cascade)** — User 1:N Todo 관계 + 사용자 삭제 시 Todo도 삭제

C# 문법 정리 (질문했던 것들): - **string?** 의 **?** = nullable(null 허용). DB의 NULL/NOT NULL로 번역됨. - **{ get; set; } = 값** = 기본값 초기화. null 불가 속성에 빈 값을 줘 경고 방지. - **record** = 데이터 운반용 불변 타입. 편지 메서드 불가(대신 **with** 로 복제), **new** 로 여러 개 생성 가능, 값 기반 동등성. - 관계 메서드 체인: HasOne(하나 가짐) → WithMany(여럿 가짐) → HasForeignKey(연결 컬럼) → OnDelete(삭제 동작)

6단계 — 연결문자열 + DbContext 등록

User Secrets (로컬 비밀):

```
dotnet user-secrets init --project TodoApi
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Host=...;Port=5432;Database=tododb;Username=pgadmin;Password=...;SslMode=Required" --project TodoApi
```

- `init` 이 만든 건 csproj의 `UserSecretsId` (GUID 주소표)뿐. 실제 비밀은 프로젝트 밖 `secrets.json` 에 저장 → git에 안 올라감.

Program.cs 등록:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection"));
```

- `Configuration` 이 appsettings + User Secrets + 환경변수를 자동으로 합쳐 읽음 → 코드 변경 없이 환경별 비밀 주입 가능

세 파일 구분: appsettings.json(git O, 비밀X) / csproj(git O, GUID만) / secrets.json(git X, 비밀O)

7단계 — 첫 마이그레이션 → DB 테이블 생성

```
dotnet ef migrations add InitialCreate --project TodoApi # 설계도 생성 (DB 안 건드림)
dotnet ef database update --project TodoApi # DB에 적용
```

또는 VS 패키지 관리자 콘솔: `Add-Migration InitialCreate` → `Update-Database`

- EF Core가 엔티티를 보고 `CREATE TABLE` SQL을 자동 생성·실행
- `tododb` 가 없으면 자동 생성(`CREATE DATABASE tododb`)
- `__EFMigrationsHistory` 테이블로 적용 이력 추적

마이그레이션 수정 원칙: 마이그레이션 파일을 직접 고치지 않는다. 엔티티(source of truth)를 수정 → 새 마이그레이션 생성. DB 적용 전이면 `Remove-Migration` 후 재생성, 적용 후면 새 마이그레이션 추가(누적).

8단계 — CRUD API + DTO (컨트롤러)

DTO (`Dtos/TodoDtos.cs`): `CreateTodoDto` / `UpdateTodoDto` / `TodoResponseDto` (record) - `CreateTodoDto`에 `Id` · `UserId` · `CreatedAt` 없음 = over-posting 방지 (클라이언트가 못 건드림)

`TodosController` (`Controllers/TodosController.cs`) : - `[ApiController]` — 자동 검증, 파라미터 바인딩 추론, 표준 에러 응답 - `[Route("api/[controller]")]` — `/api/todos` (클래스명 자동 치환) - DI로 `AppDbContext` 주입 - `TempUserId = 1` + 모든 쿼리에 `Where(t => t.UserId == TempUserId)` — 사용자별 분리 (★ Phase 2에서 토큰 기반으로 교체) - 5개 액션: `GetAll` / `GetById` / `Create` / `Update` / `Delete` - 반환: `Ok` (200) / `NotFound` (404) / `CreatedAtAction` (201+Location) / `NoContent` (204)

질문했던 개념들: - `Task<ActionResult<IEnumerable<TodoResponseDto>>>` = 비동기(Task) + HTTP응답(ActionResult) + 목록(IEnumerable) + DTO - `IActionResult` vs `ActionResult<T>` : 데이터 안 주면 `IActionResult`(204 등), 데이터 주면 `ActionResult`(Swagger 문서화 이점) - `FirstOrDefaultAsync` = LINQ `FirstOrDefault`의 EF Core 비동기 버전. 없으면 null → 404 처리 - `SaveChangesAsync` = Add/수정/삭제를 모았다가 한 트랜잭션으로 DB 반영. 호출 전엔 DB 안 바뀜 - `CreatedAtAction(nameof(GetById), new {id}, dto)` = `GetById`를 실행하는 게 아니라 그 URL을 만들어 Location 헤더에 담음. 본문에 생성된 id 포함 → 클라이언트는 본문 id로도 충분

9단계 — 로컬 실행 + Swagger CRUD 테스트

- 테스트용 User 1명 INSERT (FK 때문에 필요)
- `dotnet run` → `/swagger` 에서 POST→GET→PUT→DELETE 순서로 확인
- DB로 교차 확인

10단계 — 배포 (연결문자열 환경별 주입)

문제: User Secrets는 로컬에만 있음 → 클라우드 앱은 연결문자열이 없어 DB 접속 불가 **해결(B 방식)**: GitHub Secret `DB_CONNECTION_STRING` 등
록 → `deploy.yml`이 Container App secret으로 저장하고 환경변수로 참조

```
- name: Set DB connection string
run: |
  az containerapp secret set --name $CONTAINER_APP --resource-group $RESOURCE_GROUP --secrets db-conn="{{ secrets.DB_CONNECTION_STRING }}"
  az containerapp update --name $CONTAINER_APP --resource-group $RESOURCE_GROUP --set-env-vars "ConnectionStrings__DefaultConnection=secretref:db-conn"
```

- `:` → `_` (언더스코어 2개) = .NET 환경변수 규칙
- `secretref:db-conn` = 환경변수가 Container App secret을 참조 (평문 환경변수보다 안전)

5. 내가 겪은 에러와 해결 (← 핵심 학습)

에러 ① 환경 1개 제한 (7단계, Phase 0 연장)

- 증상: `MaxNumberOfGlobalEnvironmentsInSubExceeded`
- 원인: 무료 구독은 Container App 환경 구독당 1개
- 해결: 새 환경 포기, 기존 `env-deploy-test` 공유

에러 ② enum JSON 변환 실패 (9단계)

- 증상: POST 시 `400` — `The JSON value could not be converted to ... CreateTodoDto. Path: $.priority`
- 원인: DB 저장엔 문자열로 했지만(HasConversion), **API의 JSON 직렬화는 별개** → 기본은 enum을 숫자로만 받음. `"High"` 문자열을 못 받음
- 해결: Program.cs에 `JsonStringEnumConverter` 추가

```
builder.Services.AddControllers()
    .AddJsonOptions(o => o.JsonSerializerOptions.Converters.Add(
        new System.Text.Json.Serialization.JsonStringEnumConverter()));
```

- 교훈: **DB 저장 방식**과 **API 직렬화 방식**은 따로 설정해야 일관됨

에러 ③ ReflectionTypeLoadException (9단계)

- 증상: 실행 시 `Unable to load one or more of the requested types`
- 원인: **Swashbuckle.AspNetCore 10.x**(.NET 10용)가 .NET 9 프로젝트와 충돌
- 해결: `dotnet add TodoApi package Swashbuckle.AspNetCore --version 7.*` (9용으로 다운)
- 교훈: 패키지 메이저 버전을 .NET 버전과 맞춰야 함 (EF Core 때와 동일 원리)

에러 ④ dotnet-ef 버전 불일치 (4단계)

- 증상: `update` 로 다운그레이드 거부 (`9.0.17이 기존 10.0.9보다 낮습니다`)
- 원인: 글로벌 도구가 10으로 설치됨, 프로젝트는 9
- 해결: `uninstall` 후 `install --version 9.*` (update는 다운그레이드 불가)

에러 ⑤ "tododb does not exist" 처음 한 줄 (7단계)

- 증상: 마이그레이션 로그 첫 줄에 `connection fail` → 그러나 마지막은 `Done.`
- 원인: **에러가 아님**. EF가 tododb에 붙으려다 없으니 "없네 → 만들자" 하고 `CREATE DATABASE` 실행한 정상 흐름의 첫 단계
- 교훈: 로그 전체를 보고 최종 결과(`Done.`)를 확인. 중간 fail에 놀라지 말 것

에러 ⑥ UserSecretsId 없음 / push 누락 (집-회사 동기화)

- 증상: 집 PC에서 `Could not find 'UserSecretsId'`
- 원인: 회사 PC에서 Phase 1 작업을 commit/push하지 않은 채로 돔 → 집에서 pull해도 안 내려옴
- 해결: 회사 PC에서 `git add` → `commit` → `push`, 집에서 `git pull`
- 교훈: User Secrets는 PC마다 따로(git X). 작업 후 반드시 push 확인. secrets는 새 PC에서 재설정 필요

에러 ⑦ unrecognized arguments (10단계 배포)

- 증상: `az containerapp up` 단계에서 `ERROR: unrecognized arguments` + exit code 2
- 원인: `\` (줄 연결 백슬래시) + **CRLF 줄바꿈**(윈도우) 또는 줄 끝 공백 → 리눅스 러너에서 `\r` 이 명령을 깨뜨림
- 해결: `\` 줄 연결을 없애고 **명령을 한 줄로** 작성 (또는 VS Code에서 LF로 저장)
- 교훈: CI 스크립트는 줄 연결보다 한 줄이 플랫폼 간 안전. 윈도우 편집 시 CRLF 주의

6. 다른 사람들이 흔히 겪는 에러 (예방 차원)

EF Core + PostgreSQL + ASP.NET Core 조합에서 자주 보고되는 함정들. 미리 알면 디버깅이 빨라진다.

DateTime UTC 문제 (Npgsql 특유)

- 증상: `Cannot write DateTime with Kind=Local/Unspecified to PostgreSQL type 'timestamp with time zone'`
- 원인: PostgreSQL의 `timestampz` 는 UTC를 기대하는데, `DateTime.Now` (로컬)를 넣으면 거부
- 예방: `DateTime.UtcNow` 사용 (우리 엔티티는 이미 UtcNow로 작성됨). 또는 연결문자열/AppContext 스위치로 처리

마이그레이션 적용 안 됨 (DB 빈 채로)

- 증상: 앱은 뜨는데 테이블이 없음 / "relation does not exist"
- 원인: `migrations add` 만 하고 `database update` 를 안 함 (설계도만 만들고 DB 미적용)
- 예방: `add` → `update` 두 단계 모두 실행 확인

연결문자열 SSL 누락

- 증상: `no pg_hba.conf entry ... no encryption` 또는 `SSL required`
- 원인: Azure PostgreSQL은 SSL 필수인데 `SslMode=Require` 빠뜨림
- 예방: 연결문자열에 항상 `SslMode=Require`

방화벽 IP 미등록

- 증상: 연결 `timeout` 또는 "not allowed to connect"
- 원인: 접속 PC의 공용 IP가 방화벽에 없음 (유동 IP는 바뀌기도 함)
- 예방: 포털에서 현재 IP 추가. 막히면 IP 바뀐 것 의심

순환 참조 직렬화 오류 (네비게이션)

- 증상: API 응답 시 `JsonException: A possible object cycle was detected` (User→Todos→User→...)
- 원인: 엔티티를 직접 응답에 노출 → 양방향 네비게이션이 무한 순환
- 예방: **DTO로 응답**(우리 방식). 엔티티를 직접 반환하지 않으면 발생 안 함

N+1 쿼리 문제

- 증상: 목록 조회가 느림, 쿼리가 수십 개 날아감
- 원인: 관계 데이터를 반복문에서 하나씩 조회 (lazy loading 등)
- 예방: 필요한 관계는 `.Include()` 로 한 번에, 또는 `.Select()` 로 필요한 것만 투영(우리 방식)

Scoped 서비스 수명 오류

- 증상: `Cannot consume scoped service ... from singleton`
- 원인: DbContext(Scoped)를 Singleton 서비스에 주입
- 예방: DbContext는 요청당 하나(Scoped). 수명 규칙 이해

7. 면접 답변 카드

Q. EF Core의 Code First를 설명해보세요.

"엔티티 클래스를 먼저 정의하고, 마이그레이션으로 그 구조를 DB에 반영하는 방식입니다. 엔티티가 source of truth이고, 마이그레이션은 변경 diff를 자동 생성한 설계도라 직접 편집하지 않고 엔티티를 고쳐 재생성합니다."

Q. DTO를 왜 쓰나요?

"엔티티를 직접 노출하면 over-posting(클라이언트가 Id·UserId를 덮어쓰는 것)과 순환 참조 직렬화 문제가 생깁니다. DTO로 받을 필드·내보낼 필드를 제한해 보안과 안정성을 확보하고, DB 구조와 API 표면을 분리합니다."

Q. enum을 DB와 JSON에서 문자열로 처리한 이유는?

"DB 저장은 `HasConversion<string>` 으로, JSON 직렬화는 `JsonStringEnumConverter` 로 둘 다 문자열로 통일했습니다. DB를 직접 조회할 때 의미가 보이고 API 응답도 읽기 쉽습니다. 단, 숫자 저장보다 공간을 더 쓰고 enum 이름 변경 시 기존 데이터와 안 맞을 수 있는 트레이드오프가 있습니다."

Q. SaveChanges는 언제 DB에 반영되나요?

"EF Core는 Add/Remove/속성변경을 내부적으로 추적만 하다가 SaveChanges에서 하나의 트랜잭션으로 반영합니다. 따라서 호출 전까지 DB는 바뀌지 않고, 여러 변경이 원자적으로 처리됩니다."

Q. 로컬과 배포 환경의 비밀 관리를 어떻게 분리했나요?

"로컬은 User Secrets로 프로젝트 밖에 저장해 git 노출을 막고, 배포는 GitHub Secret을 CI에서 Container App secret으로 저장한 뒤 환경변수가 secretref로 참조하게 했습니다. ASP.NET Core Configuration이 환경변수를 자동으로 읽어 코드 변경 없이 환경별 주입이 됩니다."

Q. CreatedAtAction은 무엇을 하나요?

"자원 생성 시 201 Created와 함께, 생성된 자원을 조회할 수 있는 Location URL(GetById 경로)과 본문을 반환합니다. GetById를 호출하는 게 아니라 URL을 생성하는 것이며, 실제 조회는 클라이언트가 그 URL로 요청할 때 일어납니다. 본문에 id가 포함되니 프론트는 본문 id로도 충분합니다."

Q. 사용자별 데이터 분리는 어떻게 했나요?

"현재는 임시 UserId로 모든 쿼리에 `Where(t => t.UserId == userId)` 를 걸어 사용자별로 분리합니다. Phase 2에서 JWT 토큰의 claim에서 UserId를 꺼내 이 필터를 채울 예정이고, 이는 멀티테넌시의 기본 구조와 같습니다."

8. 현재 상태 & 다음 단계

완성된 것 (Phase 1)

Azure PostgreSQL Flexible Server (공용, 무료 B1ms) + 방화벽 + SSL
EF Core 9 + Npgsql, User/ToDo 엔티티 + DbContext (enum 문자열, FK Cascade, Email 유니크)
마이그레이션 → tododb에 실제 테이블 생성
ToDo CRUD API (컨트롤러) + DTO (over-posting 방지)
enum 문자열 저장 + JSON 문자열 직렬화
사용자별 쿼리 필터 (TempUserId) — Phase 2 멀티테넌시 씨앗
User Secrets(로컬) / Container App secret(배포) 분리
CI/CD로 연결문자열 주입 → 라이브 배포 (Swagger · CRUD 동작 확인)

다음: **Phase 2 (★ 면접 핵심)** - 회원가입 / 로그인 (비밀번호 해싱) - JWT 발급 · 검증 - 인증 미들웨어 (`[Authorize]`) - `TempUserId` → 토큰에서 꺼낸
실제 `UserId`로 교체 ← 03 예약 SaaS 멀티테넌시로 직결

이후 Phase 3(React), Phase 4(docker-compose + README)로 Todo 졸업 → 03 메인 프로젝트.

이 문서엔 서버 주소·계정명 등 식별자가 포함될 수 있다. 공개 레포에는 이 학습 문서를 올리지 말 것. (비밀번호·연결문자열은 절대 문서에 적지
않음)